

CloudShield AI / DAIL

05 — Domain & Data Model Specification

Version 1.0 | Final Domain/Data Baseline | 1 September 2026

Purpose: Define the canonical domain objects, data structures, identifiers, relationships, lifecycle metadata, persistence boundaries, serialization rules, and data integrity constraints used by the DAIL research core. This document translates the requirements, architecture, functional contracts, and controlled Terraform/AWS semantics into implementable data contracts.

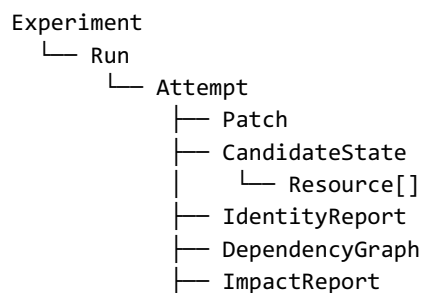
Document Control

Field	Value
Document ID	CSAI-DAIL-DATA-005
Status	FINAL DOMAIN/DATA BASELINE
Parent documents	01 SRS; 02 System Architecture; 03 DAIL Functional Specification; 04 Terraform & AWS Technical Specification
Primary runtime	Python 3.12
Persistence	SQLite initially, behind repository interfaces
Serialization	Deterministic JSON for canonical semantic data/evidence payloads
Identifier strategy	UUIDs for records; stable Terraform addresses/resource fingerprints for semantic identity
Primary objective	Make state, invariant, analysis, verification, promotion, and evidence data explicit and traceable.

1. Domain Model Goals

- Represent every meaningful DAIL concept explicitly.
- Separate domain identity from database identity.
- Keep Candidate State and Trusted State distinguishable in the data model.
- Make evidence state-specific and version-aware.
- Represent uncertainty rather than collapsing it into optimistic values.
- Allow every analysis artifact to be traced to a candidate, parent state, run, and configuration.
- Support deterministic serialization and reproducible experiments.
- Permit SQLite initially without coupling the domain model to SQLite.

2. Domain Model Overview



- └─ VerificationReport
- └─ PromotionDecision
- └─ EvidenceEvent[]

TrustedState

- └─ Resource[]
- └─ ProtectedInvariant[]
- └─ EvidenceReference[]

IndependentOracleResult

- └─ references Run/Attempt/Candidate but is separate from DAIL decision evidence

3. Identifier Strategy

Identifier	Purpose	Example
record_id	Unique persistent record identity.	UUID
state_id	Identity of a state record.	UUID
state_version	Monotonic version within a lineage/context.	42
candidate_id	Identity of candidate evaluation object.	UUID
resource_address	Terraform source address.	aws_instance.app
resource_fingerprint	Deterministic semantic fingerprint.	sha256(...)
invariant_id	Stable identity of protected obligation.	INV-FUNC-001
invariant_version	Version of invariant definition.	1
run_id	Experiment execution identity.	UUID
attempt_id	Repair/evaluation attempt identity.	UUID
evidence_id	Immutable evidence record identity.	UUID
decision_id	Promotion decision identity.	UUID

4. Common Data Types

4.1 Enumerations

Type	Allowed values
CandidateSource	FIXED_PATCH, LLM
StateKind	TRUSTED, CANDIDATE
IdentityStatus	SAME, DIFFERENT, UNCERTAIN
ChangeType	UNCHANGED, MODIFIED, CREATED, DELETED, REPLACED, UNCERTAIN
VerificationResult	PASS, FAIL, UNKNOWN, UNSUPPORTED, VERIFIER_ERROR
DecisionType	PROMOTE, REJECT, RETRY, ESCALATE
InvariantStatus	REGISTERED, VERIFYING, PROTECTED, AFFECTED, REVERIFYING, VIOLATED, UNCERTAIN
ImpactStatus	UNAFFECTED, AFFECTED, UNCERTAIN
ResourceSupport	SUPPORTED, UNSUPPORTED, UNKNOWN
EvidenceKind	INPUT, NORMALIZATION, IDENTITY, DEPENDENCY, IMPACT, VERIFICATION, DECISION, STATE_COMMIT, ERROR, ORACLE, LLM

4.2 Timestamps

All persisted event timestamps shall be timezone-aware and stored in a consistent canonical format. UTC is recommended for persistence; presentation may convert to local time.

4.3 JSON

JSON is used for flexible semantic payloads where a rigid relational column set would create unnecessary coupling. Any JSON used for fingerprints or experimental evidence must use deterministic key ordering and a documented serialization policy.

5. Resource Domain Model

5.1 Resource

```
Resource {
  record_id: UUID
  address: string
  resource_type: string
  logical_identity: JSON
  attributes: JSON
  security_attributes: JSON
  semantic_attributes: JSON
  references: Reference[]
  provenance: Provenance
  canonical_json: string
  fingerprint: string
  support_status: ResourceSupport
}
```

5.2 Resource Identity Rules

- `record_id` is database/domain-record identity and must not be used as semantic Terraform identity.
- `address` is source-level identity within a configuration context.
- `fingerprint` represents canonical supported semantic content.
- Provider/runtime IDs may be stored in `logical_identity` or `provenance` where available.
- Runtime IDs must not automatically become stable semantic identity if the resource can be replaced.

5.3 Reference

```
Reference {
  source_address: string
  source_attribute: string
  target_address: string
  reference_type: string
  resolution_status: SUPPORTED | UNKNOWN | UNSUPPORTED
}
```

6. Provenance Model

```
Provenance {
  source_kind: TERRAFORM_CONFIG | TERRAFORM_PLAN | TERRAFORM_STATE | AWS_API | LLM | DERIVED
  source_reference: string
  tool_name: string
  tool_version: string
  parser_version: string
}
```

```

    normalization_version: string
    timestamp: datetime
}

```

Provenance is not decorative metadata. It is required to explain where a semantic fact originated and to support reproducibility.

7. Trusted State Model

7.1 Canonical Structure

```

TrustedState {
    state_id: UUID
    lineage_id: UUID
    version: integer
    parent_state_id: UUID | null
    state_hash: string
    resources: Resource[]
    invariant_refs: InvariantRef[]
    evidence_refs: EvidenceRef[]
    created_at: datetime
    committed_at: datetime
    commit_decision_id: UUID
    normalization_version: string
}

```

7.2 Rules

- Only promoted candidates may become TrustedState records.
- version is monotonic within a lineage.
- state_hash is deterministic over the canonical trusted semantic representation.
- parent_state_id identifies the prior trusted state after promotion.
- Every trusted state references the invariant evidence applicable to that state.
- Historical trusted states are immutable from the domain perspective.

8. Candidate State Model

```

CandidateState {
    candidate_id: UUID
    lineage_id: UUID
    parent_state_id: UUID
    candidate_sequence: integer
    source: CandidateSource
    patch_id: UUID
    resources: Resource[]
    state_hash: string
    construction_status: BUILDING | READY | FAILED
    created_at: datetime
    normalization_version: string
}

```

8.1 Rules

- CandidateState always references exactly one parent TrustedState.
- CandidateState is not a TrustedState even if verification passes; it becomes trusted only through promotion.

- CandidateState is immutable after construction except for explicitly modeled processing metadata.
- Candidate state hash is calculated before analysis and remains stable for the candidate version.

9. Patch Model

```
Patch {
  patch_id: UUID
  source: FIXED_PATCH | LLM
  content: string
  format: TERRAFORM_HCL
  parent_state_id: UUID
  llm_request_id: UUID | null
  created_at: datetime
  content_hash: string
  metadata: JSON
}
```

The patch is the proposed transition input. It must be retained independently of the resulting candidate state so that the candidate can be reconstructed or audited.

10. Invariant Domain Model

10.1 Invariant Definition

```
Invariant {
  invariant_id: string
  version: integer
  name: string
  category: SECURITY | FUNCTIONAL
  description: string
  predicate: JSON
  scope: JSON
  verifier_id: string
  verifier_version: string
  status: InvariantStatus
  created_at: datetime
}
```

10.2 Invariant Reference

```
InvariantRef {
  invariant_id: string
  invariant_version: integer
  state_id: UUID
  status: InvariantStatus
  evidence_ids: UUID[]
  last_verified_at: datetime | null
}
```

The definition and the state-specific reference are intentionally separate. An invariant can retain one stable definition while its verification status/evidence changes across trusted states.

11. Dependency Graph Model

11.1 Graph

```
DependencyGraph {  
  graph_id: UUID  
  state_id: UUID  
  nodes: GraphNode[]  
  edges: GraphEdge[]  
  graph_version: string  
  created_at: datetime  
}
```

11.2 Graph Node

```
GraphNode {  
  node_id: string  
  node_type: RESOURCE | INVARIANT | SEMANTIC  
  ref_id: string  
  attributes: JSON  
}
```

11.3 Graph Edge

```
GraphEdge {  
  edge_id: UUID  
  source_node_id: string  
  target_node_id: string  
  edge_type: TERRAFORM_REFERENCE | INFRASTRUCTURE | INVARIANT_SCOPE | INVARIANT_SEMANTIC  
  status: KNOWN | UNCERTAIN | UNSUPPORTED  
  provenance: Provenance[]  
  attributes: JSON  
}
```

The graph is a derived artifact. It must be reproducible from normalized resources, supported rules, and invariant definitions.

12. Identity Report Model

```
IdentityReport {  
  report_id: UUID  
  parent_state_id: UUID  
  candidate_id: UUID  
  resource_matches: IdentityMatch[]  
  summary: JSON  
  algorithm_version: string  
  created_at: datetime  
}
```

```
IdentityMatch {  
  trusted_resource_ref: string | null  
  candidate_resource_ref: string | null  
  identity_status: SAME | DIFFERENT | UNCERTAIN  
  change_type: UNCHANGED | MODIFIED | CREATED | DELETED | REPLACED | UNCERTAIN  
  confidence: HIGH | MEDIUM | LOW | null  
  rationale: JSON  
}
```

```

    evidence_ids: UUID[]
  }

```

Confidence is explanatory metadata and must not override the explicit identity status. A LOW-confidence SAME result should be represented according to the algorithm's defined semantics; uncertainty must remain explicit.

13. Impact Report Model

```

ImpactReport {
  report_id: UUID
  parent_state_id: UUID
  candidate_id: UUID
  changed_resources: string[]
  affected_invariants: ImpactedInvariant[]
  uncertainty: JSON
  algorithm_version: string
  created_at: datetime
}

ImpactedInvariant {
  invariant_id: string
  invariant_version: integer
  status: AFFECTED | UNCERTAIN
  reasons: JSON
  dependency_path_refs: string[]
}

```

The impact report says what must be checked. It does not prove or disprove the invariant.

14. Verification Model

14.1 Verification Request

```

VerificationRequest {
  request_id: UUID
  candidate_id: UUID
  state_id: UUID
  invariant_refs: string[]
  verification_scope: JSON
  verifier_id: string
  verifier_version: string
}

```

14.2 Verification Result

```

VerificationResultRecord {
  verification_id: UUID
  request_id: UUID
  target_type: INVARIANT | SECURITY_PREDICATE | STRUCTURAL
  target_id: string
  result: PASS | FAIL | UNKNOWN | UNSUPPORTED | VERIFIER_ERROR
  evidence: JSON
  error: JSON | null
  started_at: datetime
  completed_at: datetime
}

```

14.3 Verification Report

```
VerificationReport {
  report_id: UUID
  candidate_id: UUID
  results: VerificationResultRecord[]
  summary: JSON
  verifier_set_version: string
}
```

15. Promotion Decision Model

```
PromotionDecision {
  decision_id: UUID
  candidate_id: UUID
  parent_state_id: UUID
  decision: PROMOTE | REJECT | RETRY | ESCALATE
  policy_version: string
  required_checks: JSON
  blocking_conditions: JSON
  supporting_results: UUID[]
  created_at: datetime
  committed_state_id: UUID | null
}
```

A PROMOTE decision must reference the evidence that satisfied the promotion policy. Non-promotion decisions must preserve their blocking reasons for audit.

16. Evidence Model

16.1 Evidence Event

```
EvidenceEvent {
  evidence_id: UUID
  run_id: UUID
  attempt_id: UUID
  candidate_id: UUID | null
  state_id: UUID | null
  kind: EvidenceKind
  event_name: string
  payload: JSON
  source: Provenance
  schema_version: string
  created_at: datetime
}
```

16.2 Evidence Rules

- Evidence is append-oriented.
- Evidence events should not be silently overwritten.
- Payload schemas are versioned.
- Evidence may contain references to other domain records rather than duplicating large objects.
- Sensitive values must be redacted before persistent evidence is emitted.

17. Experiment Domain Model

17.1 Experiment

```
Experiment {  
  experiment_id: UUID  
  name: string  
  protocol_version: string  
  condition_set: JSON  
  created_at: datetime  
}
```

17.2 Run

```
Run {  
  run_id: UUID  
  experiment_id: UUID  
  condition: A | B | C  
  fixture_id: string  
  fixture_version: string  
  model_config: JSON  
  tool_versions: JSON  
  started_at: datetime  
  completed_at: datetime | null  
}
```

17.3 Attempt

```
Attempt {  
  attempt_id: UUID  
  run_id: UUID  
  parent_state_id: UUID  
  attempt_number: integer  
  patch_id: UUID  
  candidate_id: UUID | null  
  decision_id: UUID | null  
  outcome: JSON  
  created_at: datetime  
}
```

18. LLM Data Model

```
LLMRequest {  
  request_id: UUID  
  attempt_id: UUID  
  model_provider: string  
  model_name: string  
  system_prompt_hash: string  
  prompt_hash: string  
  prompt_payload: JSON | redacted  
  parameters: JSON  
  created_at: datetime  
}
```

```
LLMResponse {  
  response_id: UUID  
  request_id: UUID  
}
```

```

    raw_response: JSON | redacted
    extracted_patch: string | null
    response_hash: string
    created_at: datetime
  }

```

Raw prompts/responses may be redacted or access-controlled where necessary, but experiment-relevant metadata must remain sufficient to identify the model/configuration and reproduce the condition.

19. Independent Oracle Model

```

IndependentOracleResult {
  oracle_result_id: UUID
  run_id: UUID
  attempt_id: UUID
  candidate_id: UUID
  oracle_id: string
  oracle_version: string
  target_properties: JSON
  result: JSON
  evidence: JSON
  created_at: datetime
}

```

Oracle records are separate from DAIL verification records to preserve the independence boundary.

20. Relational Persistence Model

Table	Primary key	Important relationships
experiments	experiment_id	runs
runs	run_id	experiment; attempts; evidence
attempts	attempt_id	run; patch; candidate; decision
patches	patch_id	attempt; parent state
trusted_states	state_id	lineage; parent state; decision
candidates	candidate_id	parent state; patch; attempt
resources	record_id	candidate/state ownership
invariants	invariant_id + version	state references
invariant_refs	(state_id, invariant_id, version)	trusted state; evidence
dependency_graphs	graph_id	state
identity_reports	report_id	candidate; parent state
impact_reports	report_id	candidate; parent state
verification_reports	report_id	candidate
verification_results	verification_id	verification report
promotion_decisions	decision_id	candidate; parent state; committed state
evidence_events	evidence_id	run/attempt/candidate/state
llm_requests	request_id	attempt
llm_responses	response_id	request
oracle_results	oracle_result_id	run/attempt/candidate

21. Entity Relationship Overview

Experiment 1 — N Run
 Run 1 — N Attempt

Attempt 1 — N Patch/LLM events
 Attempt 1 — 0..1 Candidate
 Candidate N — 1 TrustedState(parent)
 Candidate 1 — 0..1 IdentityReport
 Candidate 1 — 0..1 DependencyGraph
 Candidate 1 — 0..1 ImpactReport
 Candidate 1 — 0..1 VerificationReport
 Candidate 1 — 0..1 PromotionDecision
 TrustedState 1 — N InvariantRef
 All major objects — N EvidenceEvent
 Run/Attempt/Candidate — N IndependentOracleResult

22. State Data Integrity Rules

Rule ID	Constraint
DATA-INT-001	Candidate.parent_state_id must reference an existing TrustedState.
DATA-INT-002	Candidate cannot reference another Candidate as its trusted parent.
DATA-INT-003	PromotionDecision.parent_state_id must equal Candidate.parent_state_id.
DATA-INT-004	A committed promotion decision must reference the newly created TrustedState.
DATA-INT-005	Promoted TrustedState.parent_state_id must equal the candidate's parent state.
DATA-INT-006	A TrustedState cannot be deleted/updated through normal application operations.
DATA-INT-007	Evidence references must point to existing records or immutable external artifacts.
DATA-INT-008	Verification result target must correspond to the candidate/state being verified.
DATA-INT-009	InvariantRef.state_id must identify the trusted state whose evidence it describes.
DATA-INT-010	State hashes/fingerprints must be reproducible from canonical content.

23. Immutability Rules

- Historical Trusted States are immutable.
- Candidate semantic content is immutable after construction.
- Patch content is immutable after creation.
- Identity/impact/verification reports are immutable results; corrections create new versioned reports rather than overwriting history.
- Promotion decisions are immutable after commitment; a later administrative action is represented as a new event.
- Evidence events are append-only.

24. Versioning Strategy

Object	Versioning
TrustedState	Monotonic state version within lineage.
Invariant	Explicit invariant definition version.

Normalization	Normalization schema/version recorded on state/resource.
Identity algorithm	Algorithm version in report.
Impact algorithm	Algorithm version in report.
Verifier	Verifier ID/version in every result.
Promotion policy	Policy version in every decision.
Evidence schema	Schema version in every evidence event.
Experiment protocol	Protocol version on experiment/run.

25. Canonical Hashing

Hashing is used for reproducibility and evidence correlation, not as a substitute for semantic comparison.

```
resource_hash = SHA256(canonical_resource_json)
state_hash    = SHA256(canonical_state_json)
patch_hash    = SHA256(canonical_patch_content)
response_hash = SHA256(canonical_or_raw_response_representation)
```

- Canonical serialization must be deterministic.
- Hash algorithms must be recorded if changed.
- Changing normalization semantics changes hashes and therefore must be versioned.

26. Repository Interfaces

```
TrustedStateRepository
- get_current(lineage_id)
- get(state_id)
- save_promoted(state)
- assert_current(state_id)
```

```
CandidateRepository
- create(candidate)
- get(candidate_id)
```

```
InvariantRepository
- get_definition(id, version)
- get_state_refs(state_id)
```

```
EvidenceRepository
- append(event)
- list_for_attempt(attempt_id)
```

```
AnalysisRepository
- save_identity(report)
- save_dependency(graph)
- save_impact(report)
- save_verification(report)
- save_decision(decision)
```

The domain services depend on these contracts rather than SQLAlchemy/SQLite-specific implementations.

27. Serialization Contract

- Domain models should serialize to explicit DTOs or schema models before persistence.
- Database IDs must not be confused with semantic addresses.
- Optional values must use explicit null semantics.
- Enum values must be stable strings.
- JSON payloads used for experiment evidence must be schema-versioned.
- Canonical semantic JSON must exclude nondeterministic timestamps, database IDs, or ephemeral runtime metadata unless those fields are intentionally part of the semantic object.

28. Data Validation

Object	Required validation
Resource	Type, address, canonical representation, support status, fingerprint.
CandidateState	Parent exists, source valid, patch reference valid, hash valid.
TrustedState	Lineage/version valid, parent lineage consistent, hash valid.
Invariant	ID/version/name/category/predicate/scope/verifier present.
GraphEdge	Source/target exist, type valid, uncertainty explicit.
VerificationResult	Target valid, result valid, verifier metadata present.
PromotionDecision	Candidate/parent match, policy version present, decision valid.
EvidenceEvent	Run/attempt context and schema version present.

29. Data Handling for Unknown/Unsupported

Unknown and unsupported are domain states, not missing database values.

```
support_status = SUPPORTED | UNKNOWN | UNSUPPORTED
verification_result = PASS | FAIL | UNKNOWN | UNSUPPORTED | VERIFIER_ERROR
identity_status = SAME | DIFFERENT | UNCERTAIN
dependency_status = KNOWN | UNCERTAIN | UNSUPPORTED
```

The data model must preserve these distinctions so downstream policy can make conservative decisions.

30. Example Canonical Trusted State

```
{
  "lineage_id": "...",
  "version": 3,
  "resources": [
    {
      "address": "aws_instance.app",
      "resource_type": "aws_instance",
      "fingerprint": "sha256:..."
    },
    {
      "address": "aws_db_instance.database",
      "resource_type": "aws_db_instance",

```

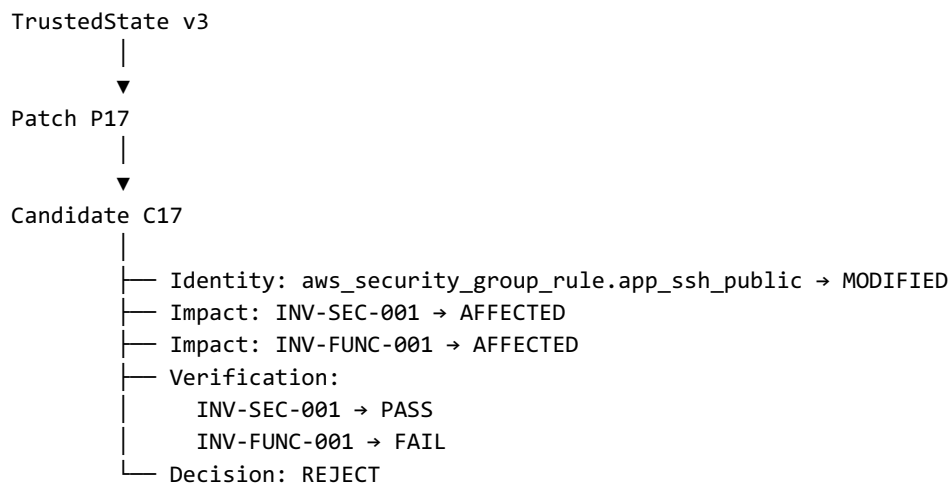
```

    "fingerprint": "sha256:..."
  }
],
"invariants": [
  {"id": "INV-FUNC-001", "version": 1, "status": "PROTECTED"}
]
}

```

This example illustrates the structure only; production fixtures must use the exact schema implementation and controlled identifiers.

31. Example Candidate Transition Data



The rejected candidate retains all records but produces no TrustedState v4.

32. Database Transaction Boundaries

Promotion is the critical transaction boundary.

```

BEGIN TRANSACTION
  assert current TrustedState = expected parent
  persist PromotionDecision
  persist new TrustedState
  persist state/invariant evidence references
COMMIT

```

- If any critical write fails, the transaction must roll back.
- Evidence that is external to the transaction may use a pending/failed marker and must never imply successful promotion when the commit failed.
- The exact database transaction implementation is owned by the persistence layer, but the domain invariant is non-negotiable.

33. Data Retention and Experiment Integrity

- Retain raw candidate patches for every experiment attempt.
- Retain decision evidence for both promoted and rejected candidates.
- Retain verifier metadata sufficient to reproduce the result.
- Retain independent oracle outputs separately.
- Do not delete rejected candidates merely because they were unsuccessful.
- Cleanup of cloud resources must not delete research records.

34. Data Security

- Never store cloud credentials in domain records.
- Redact secrets from Terraform, LLM, AWS, and verifier payloads where appropriate.
- Restrict access to raw LLM prompts/responses if they contain sensitive context.
- Do not store unnecessary personally identifying information in experiment records.
- Database backups/artifacts must be handled according to project security policy.

35. Domain-to-Architecture Mapping

Domain object	Owning component
CandidateState	Candidate State Builder
TrustedState	Trusted State Manager
Invariant / InvariantRef	Invariant Store
Resource	Terraform Normalization
Reference	Dependency Engine
DependencyGraph	Dependency Graph Engine
IdentityReport	Identity Engine
ImpactReport	Impact Engine
VerificationReport	Verification Engine
PromotionDecision	Promotion Controller
EvidenceEvent	Evidence Pipeline
LLMRequest/Response	LLM Adapter
IndependentOracleResult	Independent Oracle boundary

36. Data Model Acceptance Criteria

- All core DAIL concepts have explicit domain models.
- Candidate and Trusted State cannot be confused by schema.
- Parent-state lineage is mandatory and validated.
- Invariant definitions are separated from state-specific verification references.
- Identity, impact, verification, and promotion reports are independently persisted.
- UNKNOWN, UNSUPPORTED, UNCERTAIN, and VERIFIER_ERROR are explicit domain values.
- Evidence is append-oriented and versioned.
- State/resource/patch hashes are deterministic.
- Promotion transaction constraints prevent partial trusted-state advancement.
- SQLite implementation can be replaced without changing domain semantics.

37. Implementation Guidance

Use a schema-validation layer such as Pydantic or equivalent for domain/DTO validation if consistent with the project's final technology decision. SQLAlchemy models should remain persistence mappings rather than becoming the application's semantic domain model.

Keep JSON payloads bounded and typed where possible. Flexible JSON should be used for truly variable evidence/configuration rather than as a substitute for core domain schemas.

38. Non-Goals

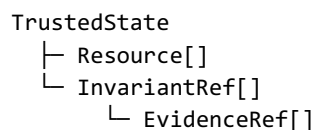
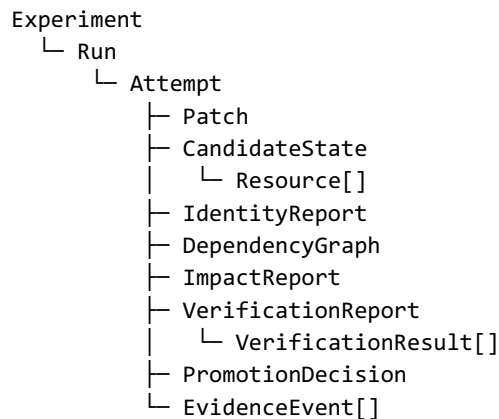
- Defining the exact identity algorithm.
- Defining the exact dependency traversal algorithm.
- Defining the complete verification predicates.
- Defining the final promotion matrix.
- Implementing every Terraform/AWS resource.
- Creating a distributed event-sourcing platform.
- Using the database schema as a replacement for domain logic.

39. Final Domain/Data Directive

The DAIL data model must make trust, lineage, uncertainty, verification evidence, and promotion authority explicit in the structure of the data. If a critical distinction exists in the functional model, it must not be collapsed into a generic string, null, boolean, or overloaded record.

The domain model is the contract shared by the DAIL engines. Later specifications may refine fields and constraints, but they must preserve the semantic distinctions defined here.

Appendix A — Core Object Graph



LLMRequest → LLMResponse

IndependentOracleResult — separate from DAIL verification

Appendix B — Next Document Dependencies

Next document	What this data model provides
06 — State & Invariant Lifecycle	TrustedState, CandidateState, Invariant, InvariantRef, evidence references.
07 — Identity & Dependency	Resource, Reference, IdentityReport, DependencyGraph.
08 — Impact & Invalidation	ImpactReport, ImpactedInvariant, graph relationships.
09 — Verification & Promotion	VerificationRequest/Result/Report, PromotionDecision.
10 — LLM Integration	Patch, LLMRequest, LLMResponse.
11 — Evidence & Logging	EvidenceEvent and provenance.
12 — Testing & QA	Fixtures and deterministic domain objects.

Appendix C — Baseline Authority

This specification formalizes the data contracts implied by Documents 01–04. It preserves the project's deterministic-first DAIL architecture, controlled Terraform/AWS model, explicit trust boundary, protected invariant semantics, conservative uncertainty handling, independent oracle separation, and evidence-driven experimentation.